INDEX

Name S. Mithulesh Class ..... Div. ..... Roll No 240701314

School Name REC Sub. Computer Network Observation

S. No.	Date	Particular	Page No.	Signature
1.		Study of various network command used in Linux & windows	1	
2.		study of Different types of Comp network cables	5	
3.		CISCO PACKET TRACER SIMULATION	9	
4.		Setting up a LAN	11	
5.		Packet capture tool: wireshark	13	
6.		Python Implementation of Hamming code Concept	15	
7.		sliding window Protocol	17	
8		N-Map host discovery	19	
9.		subnetting	21	
10a)		Internetworking with Routers	23	
10b)		Internetworking using wireless Router	25	
11a)		Static Routing	27	
11.b)		RIP (Routing information Protocol)	29	

GAMING UNLIMITED

Index	Experiment name	Pg no	Sign
12	End to End communication at transport layer	31	
13.	Implement your own ping program	33	
14.	Packet sniffing using RAW socket	35	

Completed

Exp: 1

1) arp -a
show
IP addres
o/p
Interface
Internet
19 2 168
244-00
23-9-2

2) hostname
o/p :

3) ip config
Information
ethernet
o/p:
windo
Host
pr
N

4) nb stat
TCP/IP
output/
NBTSI

Sign

Exp:1 Study of various network command used in Linux and window

1.) arp -a : show IP Address of your computer along with IP addres and MAC address of your router

o/p

Interface 192.168.156.1 ... 0x7

Internet Address	physical address	type
19.2.168.56.255	f.f.ff.ff.ff.	static
244.00.22	01.00.5e.00.02	static
23.9.255.25.250	01.00.5e.ff.ff.ff	static

2) hostname : Display name of your computer

o/p : SystemREC

3) ip config/all : Display detailed configuration on Information about TCP/IP Connection and type of ethernet adapter in the system.

o/p:

windows I/P configuration

Hostname ... systemREC

primary DNS suffix

Node type.

4) nb stat -a : Display protocol statistic and current TCP/IP connection using NBT.

output/

NBTSTAT [[-a Remote Name] [-A IP address]]

5. netstat: Display a variety of statistics about a computer active TCP/IP connection.

o/p:

Active connection

Proto	local address	Foriegn address	state
TCP	10.76.9.60:49175	10.76.96.163:domain	Time
TCP	10.7696:60.52239	142.250.250.183:http	Estab
TCP	1076.96.60.5954	10.76.96.163:domain	Time

6. nslookup: Display DNS detail such as IP Address of the particular computer.

o/p:

Server : unknown

Address : 10.76.196.163

7. pathping: Traces the root to destination address the router at 25 second test of each routes along the way.

o/p:

pathping [-g host-list][-n rn as innu-reg]

[-i address]

Linux networking commands

1) ip: Help from setting up new system and assigning IP to trouble shooting existing system

2) ip config: The ip config is utility for many system administrator's tool. held for configuring trouble shooting network.

3) mtr: MTR is program with CLI that are a network-diagnose and trouble shooting tool.

4) tcpdump: The tcpdump command is designed for capturing displays packets

5. Ping: ping is a tool that verify. Ip level connectivity to another tcp/IP computer. by sending Internet control message protocol (ICMP) echo request Message protocol (ICMP) echo request Message.

6) IP config:

flag < 73 UP loop back, RUNNING > mtu < 65536, inet 127.0.0.1 netmask 255.0.0.0 inet6 ::1 prefix len 128 scopeid 0x<10<host)

7. mtr - o/p

my traceroute [v.0.87]
local host. local domain (: 1)

key - help display mode Restart statistics
Packet

host
1::1

Loss % Snt Last
0.0% 5 2

8) tcpdump -p

listening on enp 750 of GvI

IP 93.243-167-34.b.c google.com
Content.com,

http > Oyster REC. 56518, Seq 5929, ack
35457, win 1043,

9) ping:

ping 8.8.8.8

64 bytes from 8.8.8.8 icmp, seq=1.ttl
time = 37.4 ms

Result:

Thus the various command network used on linux and window are executed successfully

Expt: 2. Study of different types of network cables

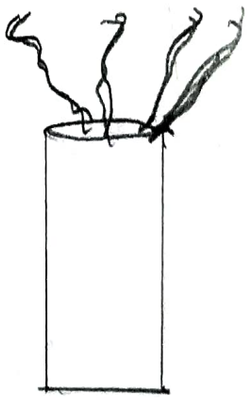
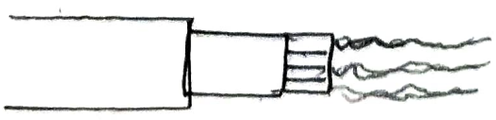
Aim:

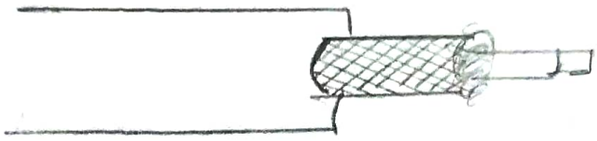
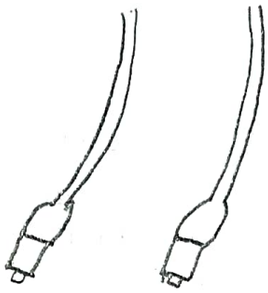
To study different types of Network cables

Different types of cables used in networking are

1) unshielded twisted pair (UTP) cable
2) shielding Twisted pair (STP) cable
3) coaxial cable
4) Fibre optic cable

cable type	category	Max data Transmission	advantage Disadvantage	Application/ use
UTP	category 3	10 bps	Advantage • cheaper • easy to install as they smaller diameter Disadvantage • More prone to (EMI) electromagnetic Interference and noise	10 base-T ethernet fast ethernet Gigabit ethernet
	category 5	upto 100 Mbps		
	category 5e	1 Gbps		
STP	category 6, 6a	10 Gbps	Advantages • shielded • Faster than UTP • Less susceptible to noise of Interfere Disadvantage • Expensive • Greater Installation effort	Gigabit ethernet 10 G ethernet (55m) widely used in data centres Gigabit ethernet 10 G ethernet (100m)
SSTP	category 7	10 Gbps		

Coaxial cable	RG-6 RG-59 RG-11	10-100 Mbps	Advantages • high bandwidth • Immune to Interference • Low loss, bandwidth Disadvantages • Limited distance cost • Cost • Size of bulky	Speed of signal upto 500 m television network High speed Internet connection
Fibre Optics cable	Single Mode Multi mode	100 Gbps	Advantages • High speed - High bandwidth • Long distance Disadvantage • Expensive • Requires skilled installation	Maximum distance of fibre optics cable is around 100 mts
UTP				
STP				
SSTP				

Coaxial cable	
Fibre optics cable.	

Result:

Thus the study of different types of networking cables is done.

Exp 3: CISCO PACKET TRACER SIMULATION

Aim:

To analyze the behavior of network devices using CISCO PACKET TRACER simulator.

Procedure:

1) From the network component box, click and drag and drop the below components
   a) 4 generic PC's and one HUB.
   b) 4 Generic PC's and one switch

2) click on connections:
   a) click on copper straight-through cable
   b) Select one of the PC and connect it to HUB using the cable. The link LED should glow in green, indicating that the link is up. Similarly connect remaining 3 PC's to the HUB.
   c) Similarly connect 4 PC's to the switch using copper straight through cable

3). click on the PC's connected to hub; go to the desktop tab, click on IP configuration and enter an IP address and subnet mask. Here, the default gateway and DNS server information is not needed as there are only two end devices in the network

click on the PDU (message icon) from the common tool bar.,

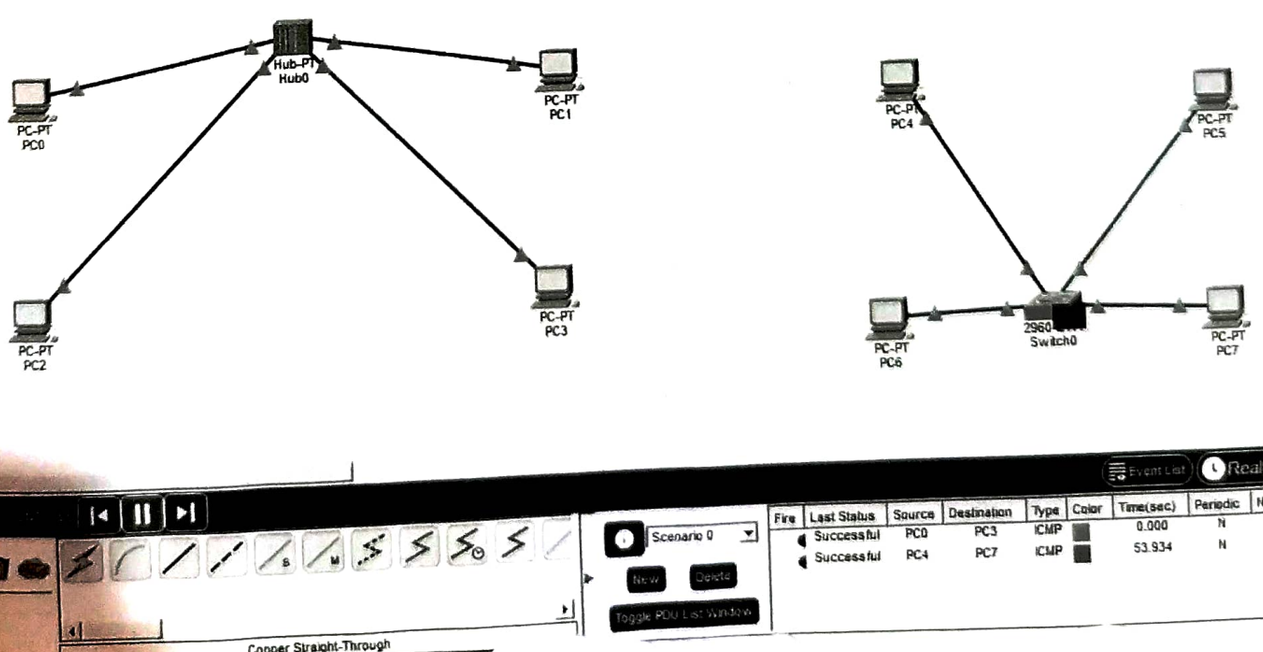
a) Drag and drop it on one of PC [source machine] and drop it on another PC [destination machine] connected to the HUB.

4) Observe the flow of PDU from source PC to destination PC by selecting the Realtime mode of simulation.

5) Repeat step 3 to step 5 for the PC's connected to the switch.

6) Observe how HUB and switch are forwarding the PDU and ~~with~~ ~~you~~ write your observation and conclusion about the behaviour of switch and HUB

Output:



Result:

Thus to analyse the behaviour of network devices using CISCO PACKET TRACER simulation is studied and implemented successfully.

Experiment - 4 Setting up a LAN

Aim:

To design, set up and configure a LAN, connecting four computers through a network, switch assign static IP addresses to all devices, verify connectivity using the PING command and demonstrate resource / data sharing across network.

Equipment required:

S.NO	Equipment	Quantity	specification
1)	Personal computer (PC)	4	Ethernet (NIC) enabled
2.	Network switch	1	8/16/24 port, unmanaged/managed
3.	Ethernet cables (cat 5e / cat 6)	4	straight - through, RJ-45 connectors.

IP address configuration table.

Device	Ip Address	Subnet Mask	connected to.
PC0	10.1.1.1	255.0.0.0	switch port Fa0/1
PC1	10.1.1.2	255.0.0.0	switch port Fa0/2
PC2	10.1.1.3	255.0.0.0	switchport Fa0/3
PC3	10.1.1.4	255.0.0.0	switchport Fa0/4
switch	10.1.1.5	255.0.0.0	switch VLAN 1 Interface.

Procedure:

step 1: place the devices;

- Open Cisco Packet Tracer.
- From Device category panel, select End device, Drag 4PCs.
- select 1 switches drag and drop (2960 series).

step 2: cable the devices;

- click connections → select copper straight-through cable
- click each PC, then click switch, select Fast-ethernet Port.
- Repeat for all four PCs (Fa 0/1 - Fa 0/4). - Green link.

step 3: Assign Ip address to PCs.

- click a PC → Desktop tab → Ip configuration.
- Enter the static IP address & subnet Mask as per the Ip table for each PC's.

Step 4: configure the Switch Management Ip

- click the switch → CLI tab, Enter the command ;- enable
  configure terminal
  Interface VLAN 1
  IP address 10.1.1.5 255.0.0.0
  no shutdown
  exit.

Step 5: verify the PC connectivity using Ping

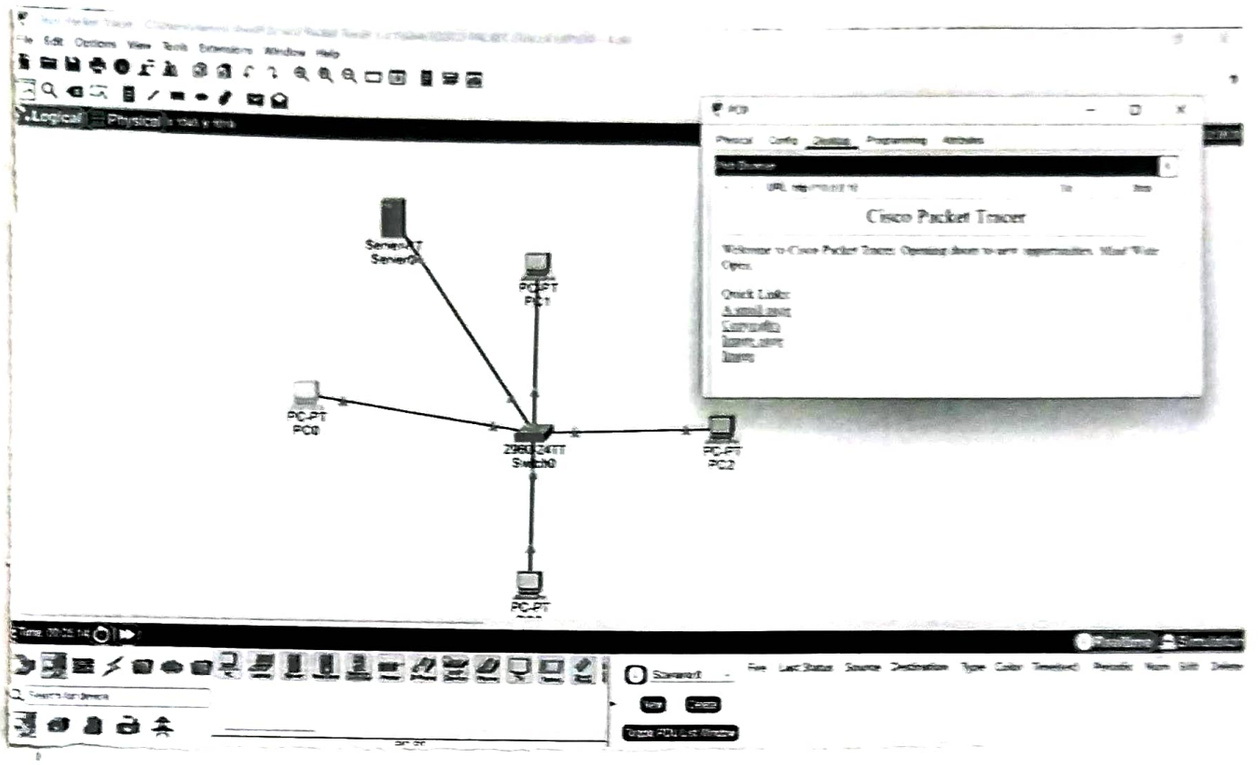
PC → Desktop → command prompt.
Ping 10.1.1.2
successfully reply - confirms connectivity

Step 6: Demonstrate data sharing (Server simulation.

- Add a server device (End devices - servers),
- Assign it an IP address via its desktop. Connect
- Open server service & enable HTTP.

Step 7: Observe packet flow in simulation mode

- Switch to simulation mode
- Send a ping or web request between two devices
  control to step through the packets



Result:

Thus the setting up of LAN, successfully verified using PING and demonstrated resource/data sharing across network.

Experiment 5 Packet Capture Tool : Wireshark

Aim:

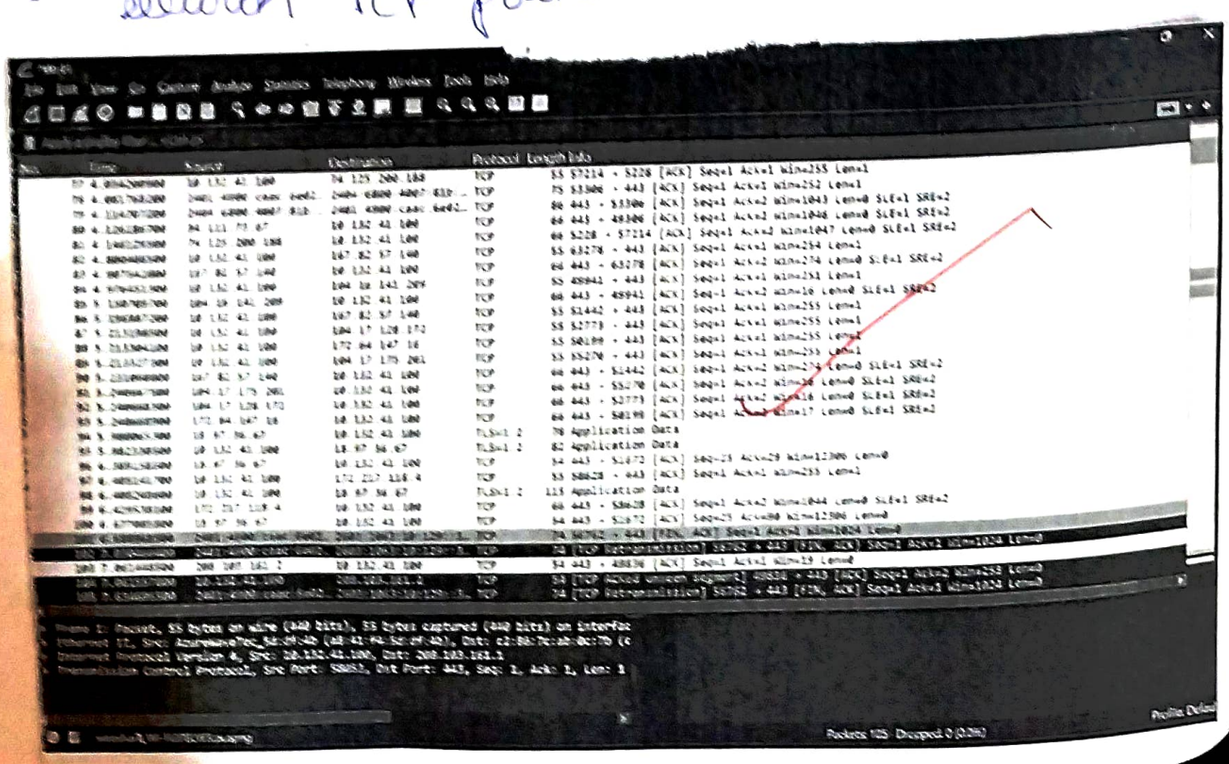
To experiment on packet capture and packet filter using wireshark..

Procedure: [capture, view, packets]

- select local area connection with ethernet
- Go to capture option.
- Select stop capture automatically after 100 packets
- start capture & save the packets.

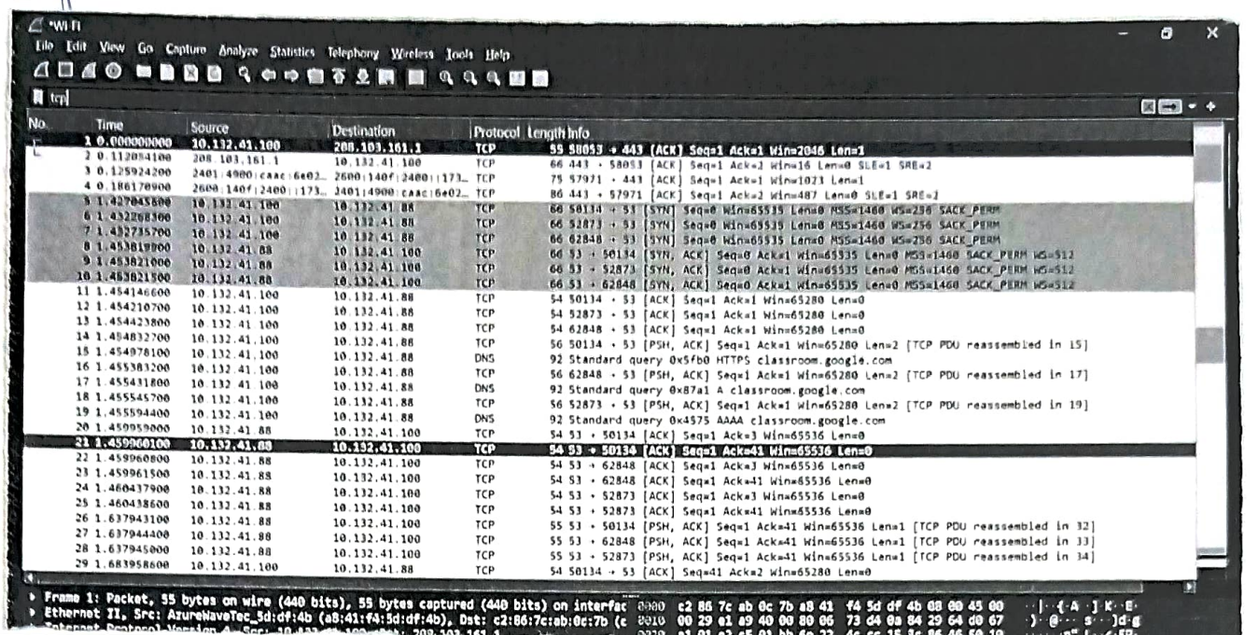
[Filter to display only TCP packets, inspects the Packets]

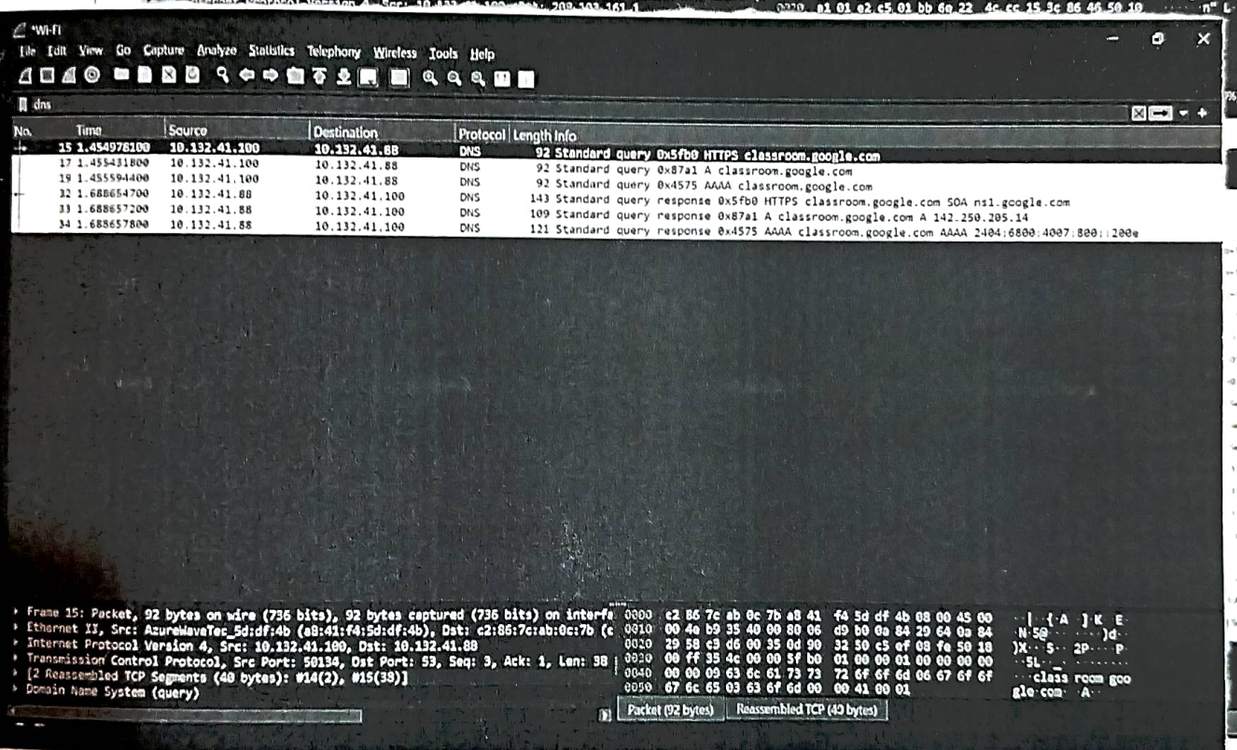
- select local area connection in wireshark
- Go to capture option
- select stop capture automatically after 100 packets
- then click start capture.
- search TCP packets in search bar.

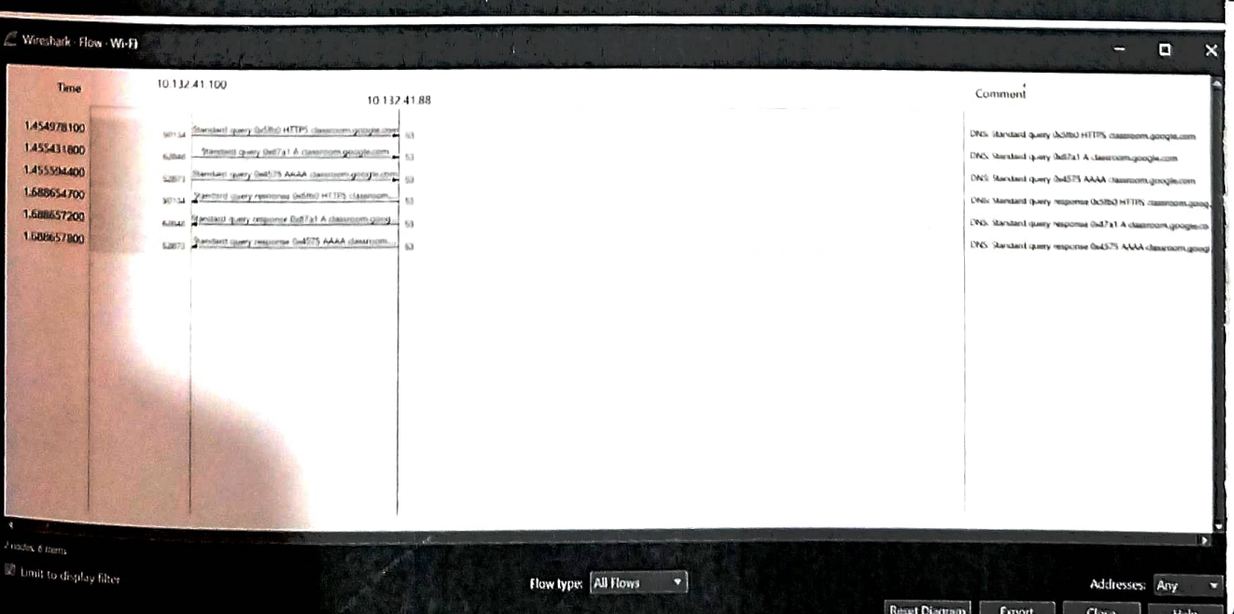


Output :-

capture Packets.







Thus the Packets are captured and filtered using Wireshark Tool successfully.

Exp: 6 python Implementation of HAMMING Code concept.

Error correction at Data link layer (AIM)

↳ Hamming code is a set a error = correction code, that can be used to detect and correct the errors when the data is transmitted from the Sender to the Receiver.

Sender. Program.

- Takes text input
- converts text to binary
- divides binary data into 4 bit groups.
- adds 3 redundant parity bits using Hamming code
- saves the encoded data in channel text.

Code:

```
def hamming-encode (data):
    encoded = " "
    for i in range (0, len(data), 4):
        d = data [i : i+4]
        if len (d) < 4:
            d = d. adjust (4, '0')
        d1, d2, d3, d4 = map(int, d)
        P1 = d1 ^ d2 ^ d4
        P2 = d1 ^ d3 ^ d4
        P4 = d2 ^ d3 ^ d4
        code = str(P1) + str(P2) + str(d1) + str(P4) + str(d2) + str(d3) + str(d4)
```

return encoded.

```
text = input("Enter text")
binary = ''.join(format(ord(c), '08b') for c in text)
print("Binary data:", binary)
encoded_data = hamming_encode(binary)
with open("channel.txt", "w") as file:
    file.write(encoded_data)
print("Hamming encoded data:",
      encoded_data);
print("Data saved to channel.txt")
```

Receiver code:

↳ Reads data from channel.txt
→ Checks every 7bit hamming block
→ Finds the error position using Syndrome
→ Corrects the errors. & removes parity bits
↳ converts binary back to ASCII text.

code :-

```
def hamming_decode(data):
    corrected = ""
    error_found = False
    for i in range(0, len(data), 7):
        block = list(map(int, data[i:i+7]))
        if len(block) < 7:
            continue
```

```
p1, p2, d1, p4, d2, d3, d4 = block
a1 = p1 ^ d1 ^ d2 ^ d4
a2 = p2 ^ d1 ^ d3 ^ d4
a4 = p4 ^ d2 ^ d3 ^ d4
error_position = a1 + (a2 * 2) + (a4 * 4)
if error_position != 0:
    error_found = True
actual_position = 1 + error_position
print("Error detected at bit position:",
      actual_position)
block[error_position - 1] ^= 1
corrected += str(block[2])
corrected += str(block[4])
corrected += str(block[5])
corrected += str(block[6])
return corrected, error_found
with open("channel_txt", "r") as file:
    channel_data = file.read().strip()
print("Data received from channel:",
      channel_data)
binary_data, error_found = hamming_decode
    (channel_data)
text = ""
```

```
for i in range(0, len(binary_data), 8):
    byte = binary_data[i:i+8]
    if len(byte) == 8:
        text += chr(int(byte,2))
print("\n corrected binary data:")
print(binary_data)
if (error_found):
    print("\n error was detected and
                                corrected:)
else:
    print("\n No error detected:)
print("Received text:", text)
```

Input:

Enter the text : HELLO

Output:

Binary data : 01001000 01000101 01001100
01001100 01001111

Hamming encoded data : 10011001 11000000 10011001
10001001 01111000 10011001
11110010 01111111

Data saved to channel.txt.

Result:

Thus the hamming code was successfully implemented to detect and correct error in the transmitted data.

Expt: 7 Sliding Window Protocol

Aim:

Write a program to implement flow control at data link layer using sliding window protocol. Simulate the flow of frames from one node to another.

Sender program.

```
import time
w = int(input("window size:"))
msg = input("message:")
frames = [(i, c) for i, c in enumerate(msg)]
i = 0
while i < len(frames):
    end = min(i + w, len(frames))
    with open("sender_buffer.txt", "w") as f:
        for n, c in frames[i:end]:
            print("Frame:", n, ":", c)
            f.write(f"{n},{c}\n")
        time.sleep(2)
with open("Receiver_buffer.txt") as f:
    ack = f.read().strip()
if ack.startswith("Ack:"):
    a = int(ack[4:])
    if a == end:
        i = end
    else:
        print("wrong Ack-Resending")
elif ack.startswith("NACK:"):
    i = int(ack[5:])
```

print ("NACK — resending frame", i)
print ("Transmission complete")

Receiver program:

```
import time
import os
expected = 0
while True:
    if not os.path.exists("sender_buffer.txt"):
        time.sleep(1)
        continue
    with open("send_buffer.txt") as f:
        frames = f.readlines()
    if not frames:
        time.sleep(1)
        continue
    for line in frames:
        n, data = line.strip().split(":")
        n = int(n)
        print("Received:", n, data)
    if n != expected:
        with open("Receiver_buffer.txt", "w")
            as f:
            f.write("NACK:" + str(expected))
        error = True
        break
        expected += 1
    if not error:
        with open("Receiver_buffer.txt", "w")
            as f:
            f.write("ACK:" + str(expected))
    open("sender_buffered.txt", "w").close()
    time.sleep(1)
```

Sender Input :-

window size : 3

Message : HELLO

Sender output :-

Frame 0 : H

Frame 1 : E

Frame 2 : L

Frame 3 : L

Frame 4 : O

Receiver Input :-

Render_Buffer.txt

0, H

1, E

2, L

3, L

4, O.

Receiver output :

Received : 0, H

Received : 1, E

Received : 2, L

Received : 3, L

Received : 4, O.

Result :

Thus Sliding window protocol was successfully implemented to control frames to handle ACKs & retransmission.

Exp : 8

N-MAP Host discovery

Aim:

To discover live host using NMAP Commands (ARP/ICMP/TCP, UDP) on the Try Hack me Platform

Introduction:

NMAP (Network Mapper) is a free and open source n/w scanning tool used to discover live hosts and Identify services running on the n/w. Host discovery is performed port scanning. to determine which systems are online. NMAP appears different host discovery techniques using ARP, ICMP, TCP and UDP.

NMAP Host discovery using ARP:

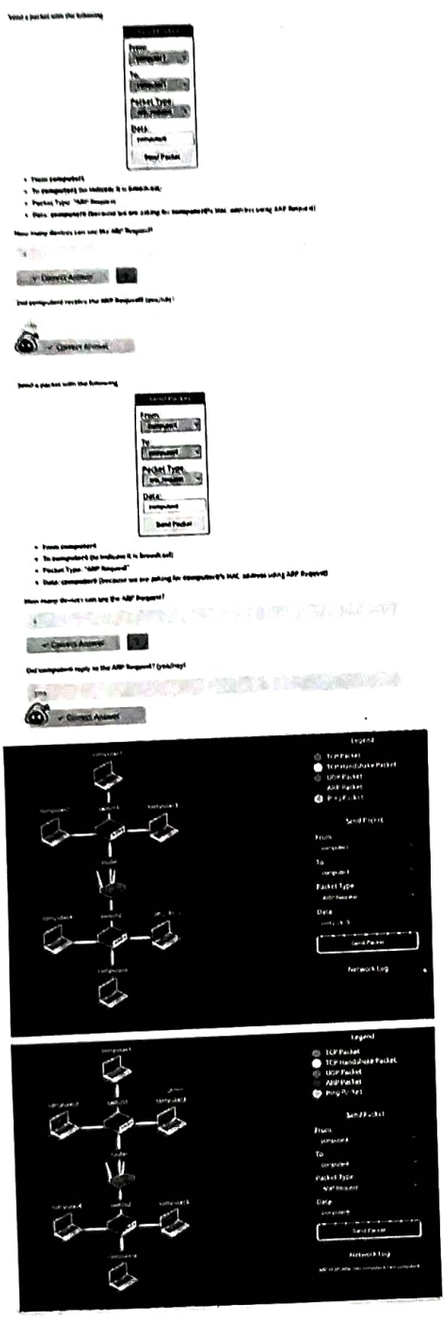
ARP is used to discover hosts on the same local subnet NMAP sends ARP request and live host respond with ARP replies. The -PR is used for ARP-based host discovery.

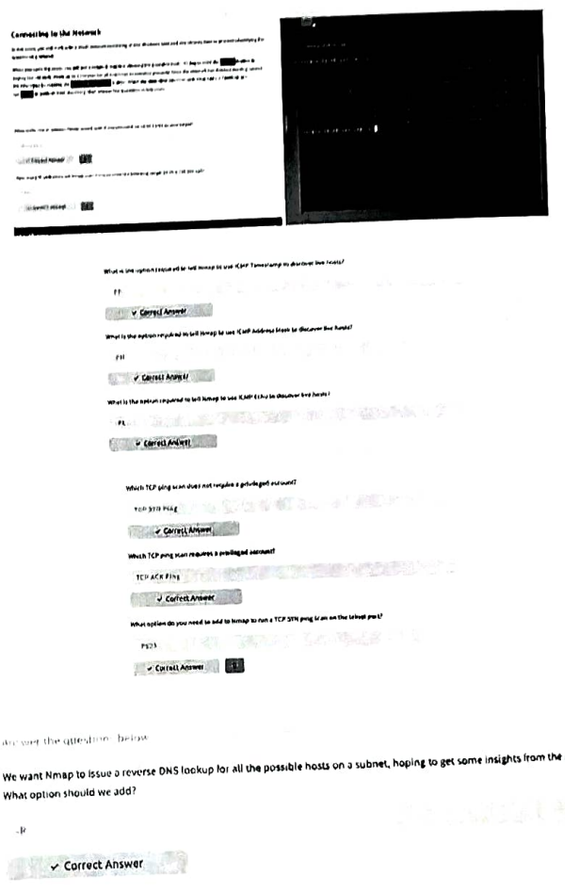
NMAP host discovery using ICMP:

ICMP Provides different req/reply mechanism for discovering live host. NMAP suggest ICMP Echo (-PE), Timestamp (-PP) and Address Mask (-PM) Scans.

NMAP Host discovery using TCP and UDP

When ICMP is filtered, NMAP can use TCP and UDP packet to determine whether a host is online. TCP host discovery includes TCP SYN Ping (-PS) and TCP ACK Ping (-PA) while UDP host discovery uses -PU.







Result:

Thus three hosts in the network were successfully discovered using NMAP host discovery techniques such as ARP, ICMP and TCP, UDP, ping scans. The different scanning methods are used to identify.

EXPT: 9

Subnetting

Aim :-

Implementation of subnetting in CISCO PACKET TRACER simulator.

Introduction:

Subnetting is the process of diving a large IP network into smaller logical network called subnets. Classless subnetting allows us to to ~~used~~ use subnet masks other than the default class-based masks, making IP address more efficient and allowing multiple networks to be created.

Here the n/w 192.168.1.0/24 is divided into smaller /27 subnets. A /27 subnets provides 30 usable host address per subnet.

Procedure:

1) open CISCO packet Tracer
2) Add required routers, switches, and PCs
3) connect the devices using appropriate cables.
4) configure the routes with the required IP addresses and subnet masks.
5) configure ~~the~~ route interface using the CLI:
   - Enable
   - configure terminal
   - Select the required Fast Ethernet / GigaBit Ethernet Interface.
   - Assign the IP address and subnet mask.
   - use no shutdown to activate the interface

6) configuration the Switch ports in access mode when Required.

7) Assign static IP address, Subnet mask and default Gateways to the PCs.

8) Use the /24 Subnet mask : 255.255.255.224.

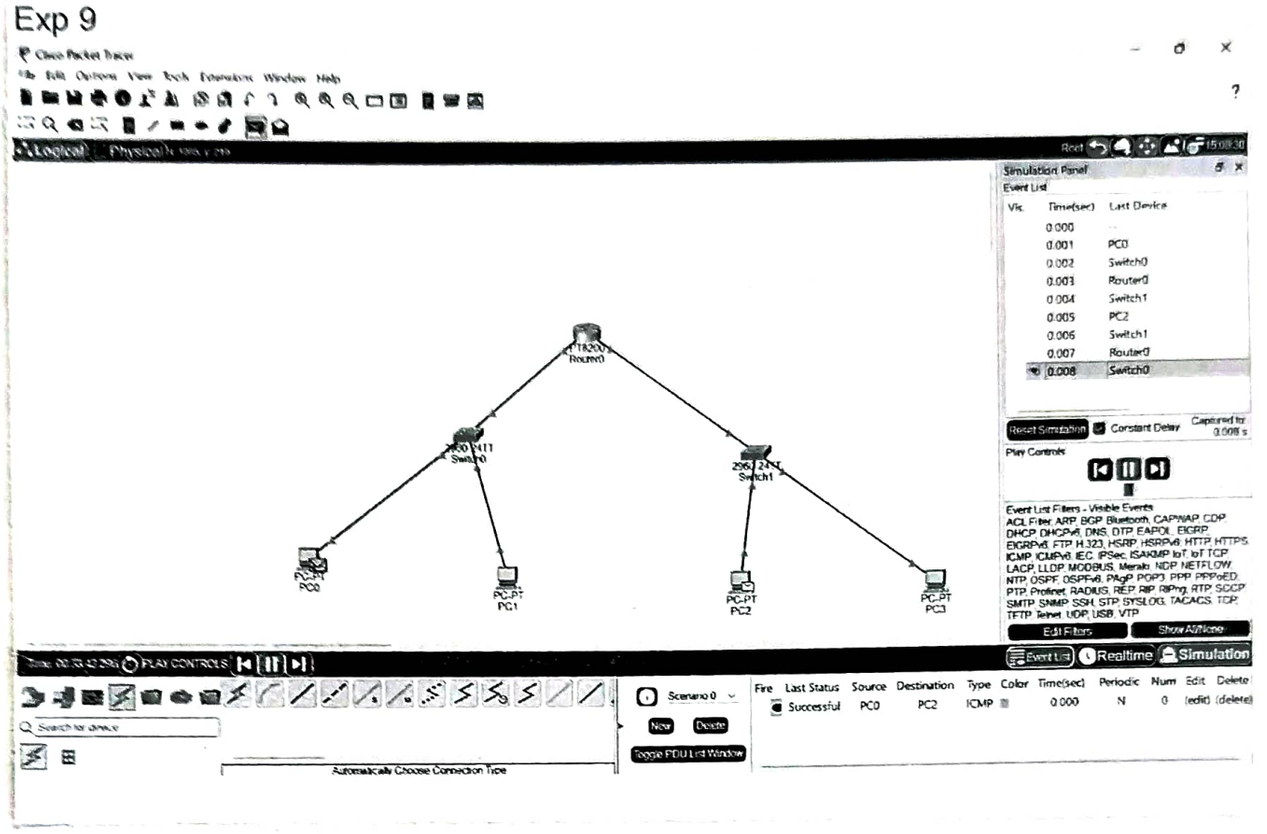
9) Configure the four Subnet :
- 192.168.1.0/27.
- 192.168.2.0/27
- 192.168.3.0/27
- 192.168.4.0/27

10) Verify the Router interface and PC's belong that respective Routers.

11) Open command prompt on the PC's

12) Use the Ping command to test connectivity between PC and respective routers.

Exp 9



Result:

Thus Subnetting was successfully implemented using CISCO PACKET TRACER.

Expt: 10(a)

Interworking with Routers.

Aim:

Interworking with routers in CISCO PACKET TRACER simulator.

Procedure:

Step 1) configure Router

i) open Cisco packet tracer and place one router and

2 PCs

ii) select the router and open the CLI and press enter

to access the command prompt.

iii) Enter privileged mode using enable.

iv) Enter global configuration mode using configure terminal

v) configure Fast-Ethernet 0 with:

- IP address: 192.168.10.1.
- Subnet mask: 255.255.255.0

vi) Use no shutdown to activate the Interface

Step 2: configure PCs

Device	IP address	Subnet mask	Default gateway
PC0	192.168.10.2	255.255.255.0	192.168.10.1
PC1	192.168.20.2	255.255.255.0	192.168.20.1

Step 3: connecting PCs with Router:

i) connect PC0 Fast ethernet 0 to Router1 Fastethernet 0

using a copper straight-through cable

ii) connect PC1 Fast Ethernet 0 to Router1 Fastethernet

1 using a copper straight-through cable

iii) wait until the Interfaces become active.

iv) use interface mode and bind a static IP for

PC0 to PC1

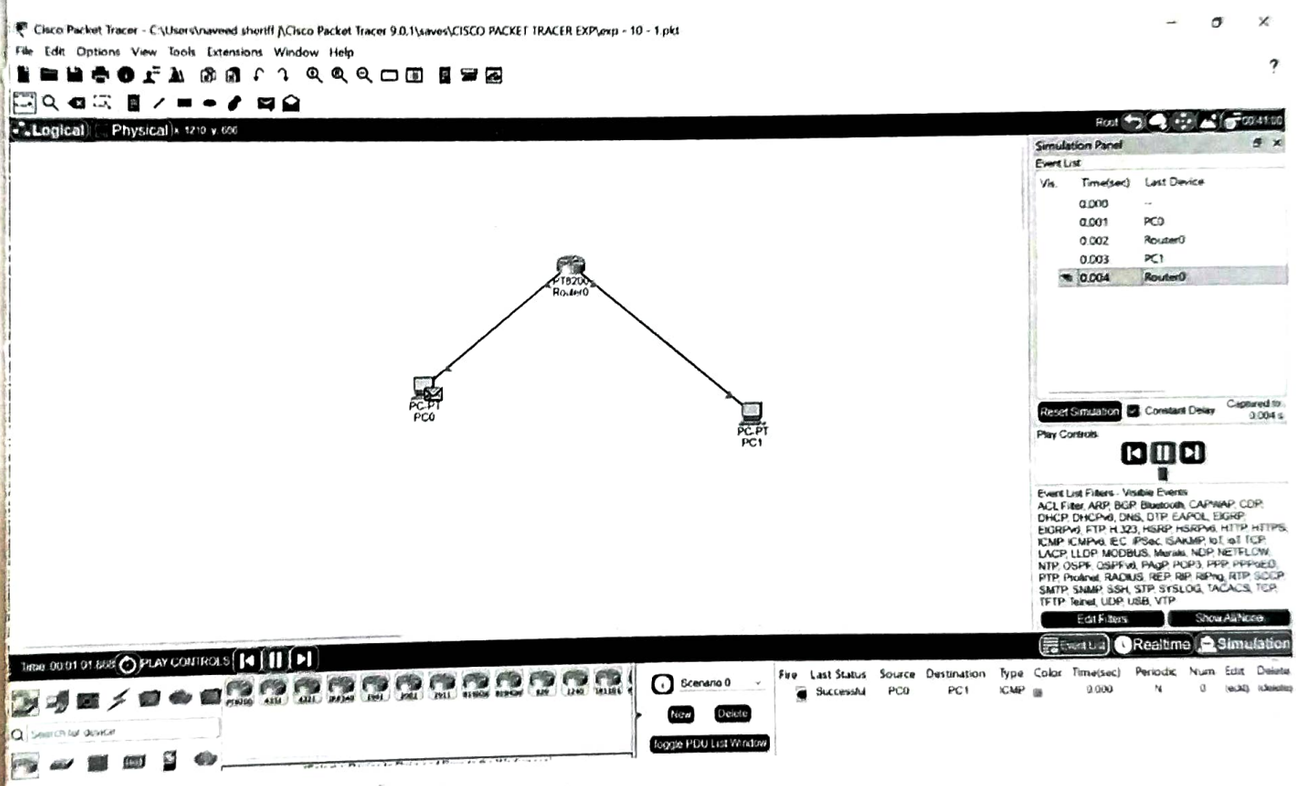
6) Verify that the PDU reaches PC1 and an acknowledge is received. Ping to PC0

Configuration TABLE :-

Routes configuration:

device	Fast/Ethernet p/c Subnet Mask	Fast ethernet 0/1	Subnet mask
Router1	192.168.10.1	255.255.255.0	192.168.20.1	255.255.255.0

Expt 10



Result:

Thus, a single internetwork was successfully designed and configured using a router in CISCO packet tracer.

Expt: 10(b)

Internetwork using wireless Routes

Aim:

To design and configure an internetwork using a wireless, DHCP Server and Internal cloud in cisco packet tracer.

Procedure:

Step 1: Build the network topology.

1) Open cisco packet Tracer and add the devices like PC, Router, laptop, cable, modem, Internet cloud and Server.
2) connect the devices using the appropriate cables.

Step 2: configure the Routes.

1) Window Router → GUI → wireless network Home → Home network.
2) Go to setup Exp and Ensure DHCP is enabled.
3) Configure the Server to 208.67.220.220.
4) Save the setting.

Step 3: configure the laptop:

1) Laptop → physical tab → power off → Remove ethernet cable and add WPC 300n wireless Module → power on the laptop
2) Desktop → PC. Select connect tab and select Homenetwork.
3) connect the laptop to the wireless network

Step 4: configure the PC

1) PC → Desktop → IP configuration. select DHCP.
2) The PC automatically receives an IP address from the wireless Router.
3) Use the command prompt and ipconfig /all to verify the configuration.

Step 5: configure the Internet cloud

1) Open the internet cloud.
2) config → cable
3) Set :- from port :- coaxial. To port :- Ethernet
4) click add.
5) Goto to ethernet and select cable or provider network.

step 6: configure the cisco web server

DHCP configuration.

Parameters	value.
Pool name	DHCP Pool
Default gateway	208.67.220.220
DNS server	208.67.220.1
Starting IP	255.255.255.0
Ethernet Mask	50.
Maximum users	

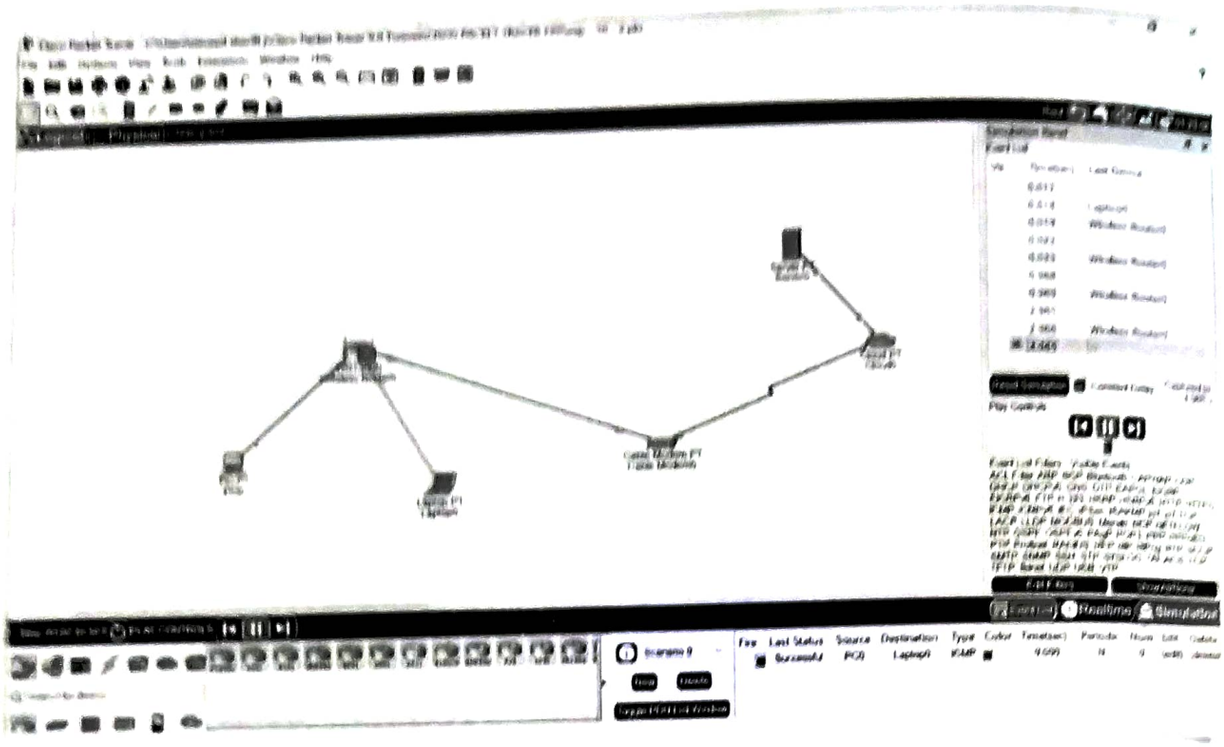
click Add

DNS configuration

Parameter	value
Name	cisco.com
Type	A. Rec
Address	203.67.220 -220

click 7:

Verify connectivity.



Result:

Thus the wireless routes, DHCP Server and internet based were successfully configured in cisco packet tracer and connectivity was verified successfully.

Expt: 11a) Static Routing:

Aim:

To simulate and configure static routing with Primary and backup routes using cisco packet tracer and verify connectivity between different networks

Procedure:-

1) Open CISCO packet Tracer and create the given topology with Router 0, Router 1, Router 2, switch and PC3.

2) configure the IP address on all the router Interface and PC's according to the given topology

3) Open the CLI of Router0 and enter configuration

4) configure static route for network 30.0.0.0/8 and 50.0.0.0/8 using router one the Primary Route.

5) configure a separate host route for 30.0.0.100/32 with Router 2 as the primary route ~~Router~~ backup route.

6) On Router1, configure route for network 10.0.0.0/8 and 40.0.0/8 with primary and backup routes.

7) On Router2, configure static route for network 10.0.0.0/8 and 30.0.0.0/8.

8) use the command: show ip route static to verify the configured static route.

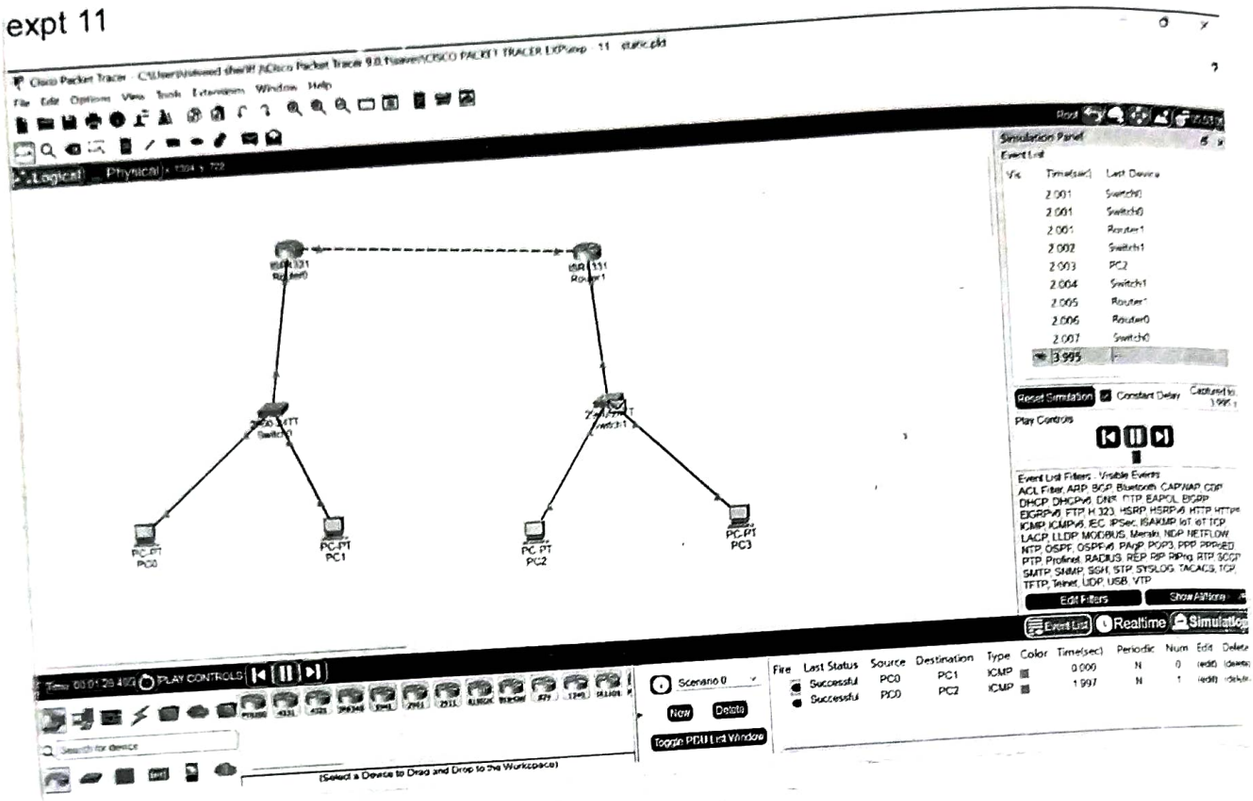
9) use the ~~Router~~ tracert command from a PC to verify the path taken by the destination network

10) Disconnect the primary link b/w R0 and R1 and use trace at again to verify that the backup route

11) static route: can be removed using: no ip route.

12) verify the routing table again using show ip route static..

expt 11



Result:

Thus the routing with primary and backup routes was successfully configured and simulated using the Cisco packet tracer.

Expt: 11(b)

RIP (Routing information protocol)

Aim:

To simulate and configure RIP (Routing Information Protocol) using Cisco Packet tracer and verify dynamic routing between different networks.

Procedure:

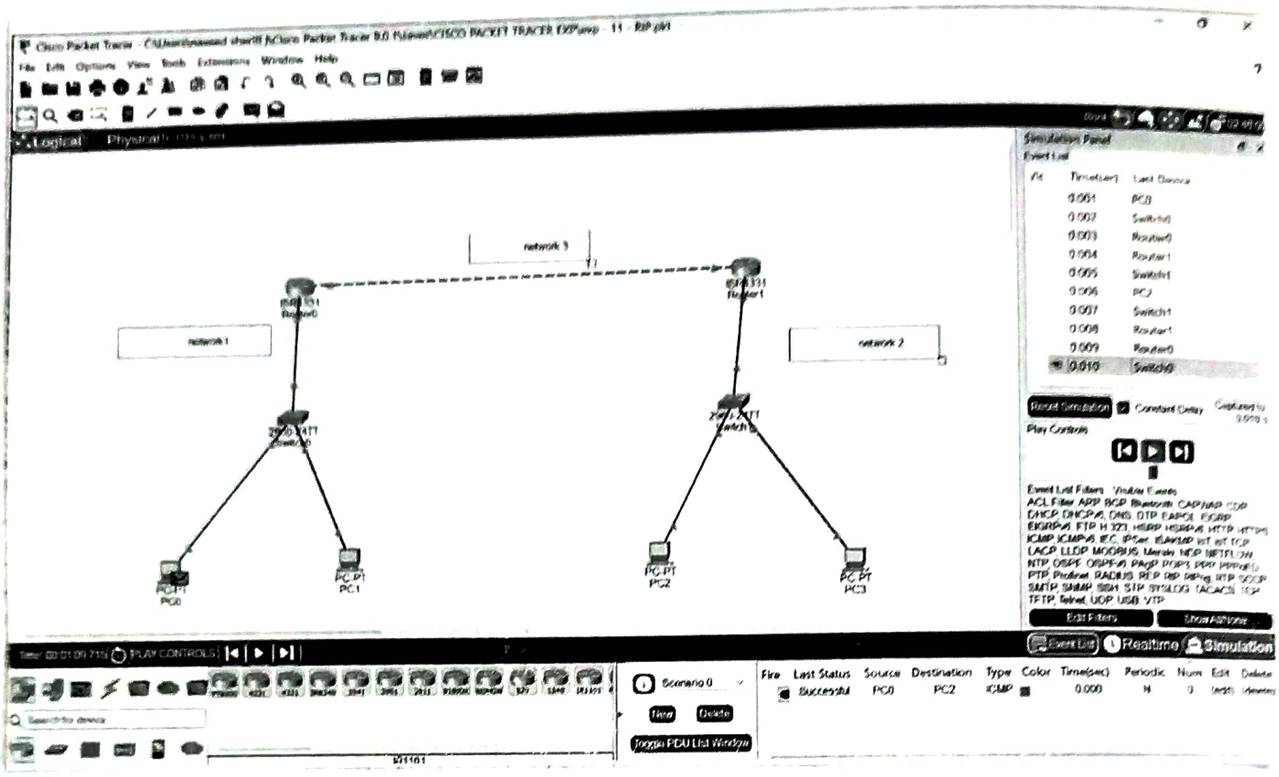
1) Open cisco packet tracer and create the given topology with Router 0, Router 1, Router 2, PC0 and PC1
2) Assign IP address to PC's and route Interfaces according to cable.
3) Configure the Fast ethernet and Serial interfaces of all routers using the assigned IP address.
4) Use No shutdown to activate all route interfaces
5) For DCE serial interfaces, configure the Required clock rate and bandwidth
6) configure RIP on R0 using:-

```
router rip
network 192.168.1.244
network 192.168.248
```

7) configure RIP on R2 using:

```
router rip
network 200.0.0.0
network 192.168.1.252
network 192.168.1.244
```

8) Allow RIP to exchange routing information automatically between the routers
10) From PC0: use the Ping command to verify connectivity
11) use the tracert command to observe the Path followed by Packet between the networks.



Result:

Thus RIP was successfully configured and simulated using cisco Packet Tracer.

Expt: 12. End to End communication at transport layer.

a) Implementation ECHO client server using TCP sockets

Aim:

To implement an Echo server application using TCP sockets.

code:

ECHO server

```
import socket
S = socket.socket()
S.bind(("localhost.", 5000))
S.listen(1)
C, addr = S.accept()
msg = C.recv(1024).decode()
print("client :", msg)
C.send(msg.encode())
C.close()
S.close()
```

ECHO CLIENT

```
import socket
S = socket.socket()
S.connect(("localhost", 5000))
msg = input("Enter message :")
S.send(msg.encode())
print("Echo :", S.recv(1024).decode())
S.close()
```

Input:

Enter message : Hello Server

Output:

client connected

Received : Hello Server. client.

Echo from Server : Hello Server.

Result:

The echo client server using TCP Sockets was studied and executed successfully.

b) Implement chat client server using TCP/IP Sockets

Aim:

To implement a chat client server application using TCP sockets

Code:

Server:

```
import socket
S = socket.socket()
S.bind(("localhost", 5000))
S.listen(1)
c, a = S.accept()
while True:
    msg = c.recv(1024).decode()
    print("Client: ", msg)
    if msg == "bye":
        break
    reply = input("Server: ")
    c.send(reply.encode())
c.close()
```

CLIENT:

```
Import socket
S= socket.socket()
s.connect(("localhost", 5000))
while True:
    msg = input ("client:")
    s.send(msg.encode())
    if msg == "Bye":
        break.
    print ("Server: ", s.recv(1024).decode())
s.close()
```

Input:

client : Hello.
Server : Hi
client : Bye

output

client : Hello
Server : Hi

Result:

The chat client server Application were successfully studied and implemented, using TCP sockets.

Exp 13: Implement your own ping program.

Aim:

To implement a ping program using python to check the connectivity between source and destination host.

Algorithm:

1) Create the destination IP address from user.
2) Send the request to destination.
3) Wait for response.
4) calculation the Response time.
5) Display whether the host is reachable.
6) Repeat the process for few Request.

CODE:

```
Import OS
import time.
host = input ("Enter host: ")
for i in range (4)
    start = time.time()
    result = os.system ("ping -n 1 " + host + " > nul")
    end = time.time()
    If result == 0:
        print ("Reply from:", host, "Time =", round
((end-start) * 1000, 2), "ms").
    else:
        print ("Request timed out")
    time.sleep(1)
```

INPUT:

Enter host: 127.0.0.1

Output:

Reply from 127.0.0.1 time = 123 ms

Reply from 127.0.0.1 time = 0.98 ms

Result:

The ping program was successfully studied and implemented.

Exp: 14

Packet Sniffing using RAW Socket.

Aim:

To write a program using RAW Sockets to capture and display network packets.

Algorithm:

1) create a RAW Socket
2) Bind the Socket to the network interface.
3) Receive packets from the network.
4) Extract the IP header Information.
5) Display the Source IP, destination IP and Protocol.
6) continue capturing packets

Code:

```
Import Socket
S = socket.socket(Socket.IPPROTO_IP, SOCKET
                  Socket.IP_HDRINCL, 1)
print("Packet sniffer started ...")
while True:
    data, addr = s.recv.from(65535)
    print("Packet received from:", addr[0])
```

Input

Network packets

Output:

Packet sniffer started...
Packet received from : 192.168.1.5
Packet received from : 192.168.1.1
Packet Received from : 192.120.183.14.

Result:

The RAW Socket Sniffing Program was successfully implemented and network packets were captured and displayed.